

**Alunos:** Henrique Leão e Rhuan Victor

## Prática 4 - UDP, datagramas, timeout e retransmissão simples

### Execução dos códigos

#### 1 - Servidor e Cliente na mesma máquina

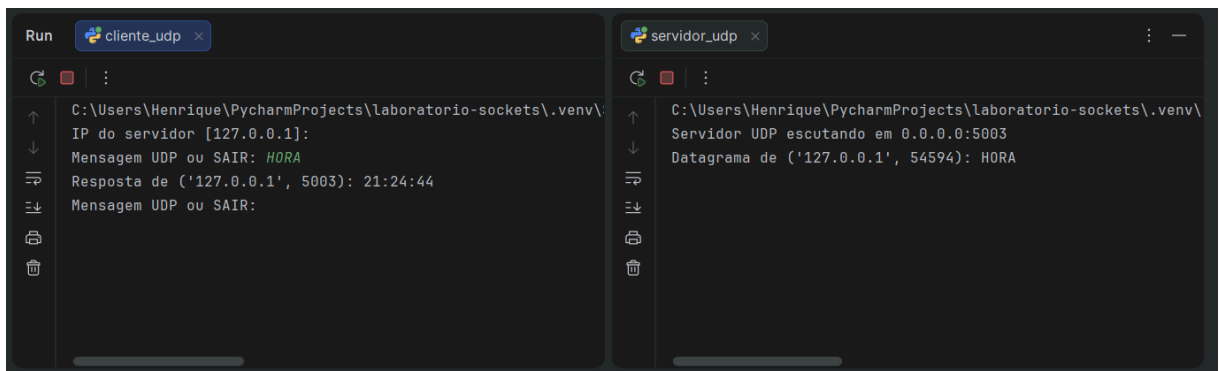


The image shows two terminal windows side-by-side. The left window, titled 'cliente\_udp', shows the client sending a message 'HORA' to the server at IP 127.0.0.1:5004. The server responds with the time '21:23:10'. The right window, titled 'servidor\_udp', shows the server listening on 0.0.0.0:5003 and receiving a datagram from '127.0.0.1' at port 54590 with the message 'HORA'.

```
Run cliente_udp x
C:\Users\Henrique\PycharmProjects\laboratorio-sockets\.venv\
IP do servidor [127.0.0.1:5004]:
Mensagem UDP ou SAIR: HORA
Resposta de ('127.0.0.1', 5003): 21:23:10
Mensagem UDP ou SAIR: |

Run servidor_udp x
C:\Users\Henrique\PycharmProjects\laboratorio-sockets\.venv\
Servidor UDP escutando em 0.0.0.0:5003
Datagrama de ('127.0.0.1', 54590): HORA
```

#### 2 - Execução do Cliente com o Servidor Desligado

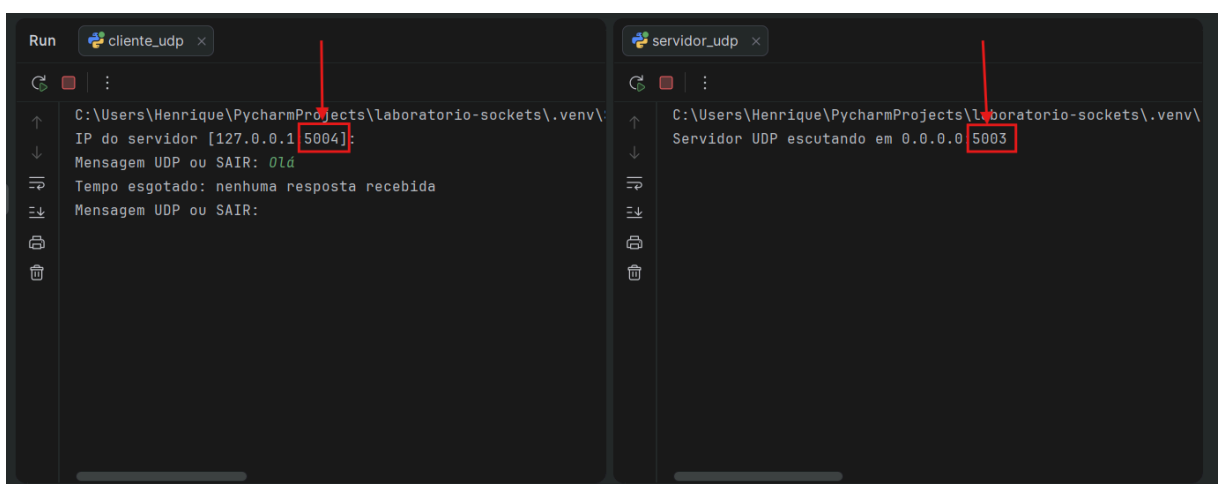


The image shows two terminal windows. The left window, 'cliente\_udp', shows the client sending a message 'HORA' to the server at IP 127.0.0.1:5004. The server responds with the time '21:24:44'. The right window, 'servidor\_udp', shows the server listening on 0.0.0.0:5003 and receiving a datagram from '127.0.0.1' at port 54594 with the message 'HORA'.

```
Run cliente_udp x
C:\Users\Henrique\PycharmProjects\laboratorio-sockets\.venv\
IP do servidor [127.0.0.1:5004]:
Mensagem UDP ou SAIR: HORA
Resposta de ('127.0.0.1', 5003): 21:24:44
Mensagem UDP ou SAIR:

Run servidor_udp x
C:\Users\Henrique\PycharmProjects\laboratorio-sockets\.venv\
Servidor UDP escutando em 0.0.0.0:5003
Datagrama de ('127.0.0.1', 54594): HORA
```

#### 3 - Cliente tenta conectar na porta errada



The image shows two terminal windows. The left window, 'cliente\_udp', shows the client sending a message '01d' to the server at IP 127.0.0.1:5004. The server responds with the time '21:24:44'. The right window, 'servidor\_udp', shows the server listening on 0.0.0.0:5003 and receiving a datagram from '127.0.0.1' at port 54594 with the message '01d'.

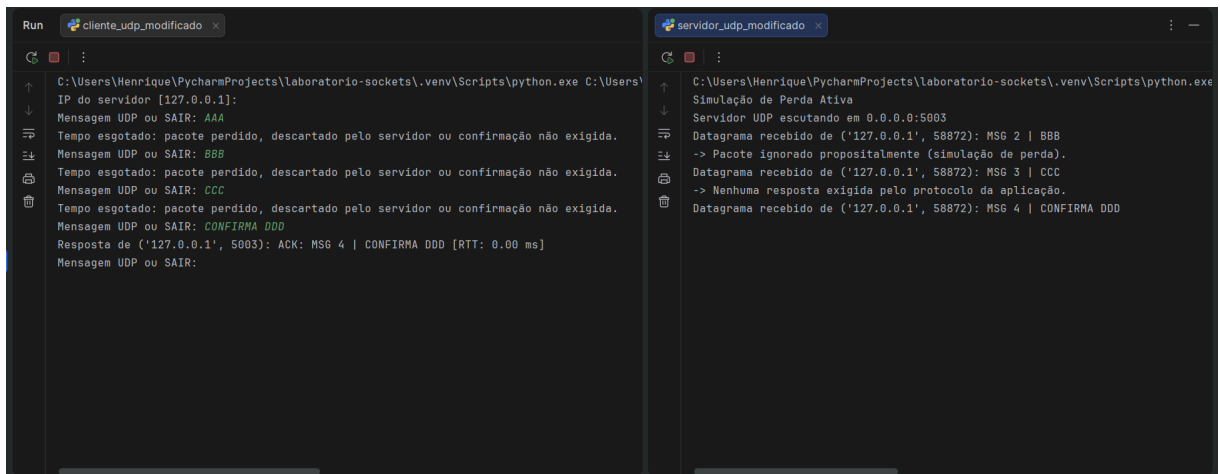
```
Run cliente_udp x
C:\Users\Henrique\PycharmProjects\laboratorio-sockets\.venv\
IP do servidor [127.0.0.1:5004]:
Mensagem UDP ou SAIR: 01d
Tempo esgotado: nenhuma resposta recebida
Mensagem UDP ou SAIR:

Run servidor_udp x
C:\Users\Henrique\PycharmProjects\laboratorio-sockets\.venv\
Servidor UDP escutando em 0.0.0.0:5003
Datagrama de ('127.0.0.1', 54594): 01d
```

**TCP: Erro imediato e ativo.** A falha ocorre instantaneamente na tentativa de conexão (`connect()`), lançando `ConnectionRefusedError`, pois o protocolo exige e não consegue realizar o *handshake* inicial.

**UDP: Erro silencioso e passivo.** O envio (`sendto()`) ocorre com sucesso às cegas. A aplicação só nota a falha na leitura (`recvfrom()`), dependendo de um timeout manual para não ficar bloqueada infinitamente (ou capturando a rejeição de porta do SO, como o `ConnectionResetError` no Windows).

#### 4 - Execução das Variações



```
Run cliente_udp_modificado x
C:\Users\Henrique\PycharmProjects\laboratorio-sockets\.venv\Scripts\python.exe C:\Users\
IP do servidor [127.0.0.1]:
Mensagem UDP ou SAIR: AAA
Tempo esgotado: pacote perdido, descartado pelo servidor ou confirmação não exigida.
Mensagem UDP ou SAIR: BBB
Tempo esgotado: pacote perdido, descartado pelo servidor ou confirmação não exigida.
Mensagem UDP ou SAIR: CCC
Tempo esgotado: pacote perdido, descartado pelo servidor ou confirmação não exigida.
Mensagem UDP ou SAIR: CONFIRMA DDD
Resposta de ('127.0.0.1', 5003): ACK: MSG 4 | CONFIRMA DDD [RTT: 0.00 ms]
Mensagem UDP ou SAIR:

servidor_udp_modificado x
C:\Users\Henrique\PycharmProjects\laboratorio-sockets\.venv\Scripts\python.exe
Simulação de Perda Ativa
Servidor UDP escutando em 0.0.0.0:5003
Datagrama recebido de ('127.0.0.1', 58872): MSG 2 | BBB
-> Pacote ignorado propositalmente (simulação de perda).
Datagrama recebido de ('127.0.0.1', 58872): MSG 3 | CCC
-> Nenhuma resposta exigida pelo protocolo da aplicação.
Datagrama recebido de ('127.0.0.1', 58872): MSG 4 | CONFIRMA DDD
```

## Respostas de Análise

**1. Por que o servidor UDP não usa listen() nem accept()?**

O protocolo UDP não estabelece conexão prévia antes do envio dos dados. O servidor simplesmente recebe datagramas variados em uma mesma porta sem precisar de circuitos virtuais ou *handshakes* que demandariam conexões individuais persistentes como no TCP.

**2. O que recvfrom() retorna além dos dados?**

A função `recvfrom()` retorna os dados recebidos e também o endereço (IP e porta) do remetente.

**3. Por que o cliente precisa de timeout em UDP?**

Porque o protocolo UDP não garante a entrega, a ordem ou a ausência de duplicidade dos datagramas. Sem um timeout, a função bloqueante de leitura ficaria travada ao infinito caso o pacote de requisição ou a resposta do servidor se perdessem durante o trânsito da rede.

**4. A retransmissão simples implementada garante entrega? Justifique.**

Não. Ao contrário do TCP, que lida com a entrega no nível de transporte, o UDP delega o tratamento de confirmações para a aplicação. A retransmissão simples com três tentativas apenas eleva a estatística de sucesso. Caso a rede continue inoperante ou o destino em falha por tempo superior ao das tentativas, a informação será totalmente descartada.

**5. Em que situações UDP pode ser preferível ao TCP?**

Em aplicações que necessitam de baixa latência e que toleram a perda parcial de pacotes, onde refazer a transmissão de um dado antigo atrasaria dados mais recentes e relevantes. Exemplos de uso clássicos abrangem DNS, transmissão de *streaming* de mídia, jogos e serviços de telemetria.